

Especificação de Requisitos de Software com Casos de Uso Parte 2

Profa. Dra. Lina Garcés

O que aprenderá na aula?

01

Casos de Uso

02

Relações dos

UC

Herança
Include
Extend

03

Descrição
Textual de UC

04

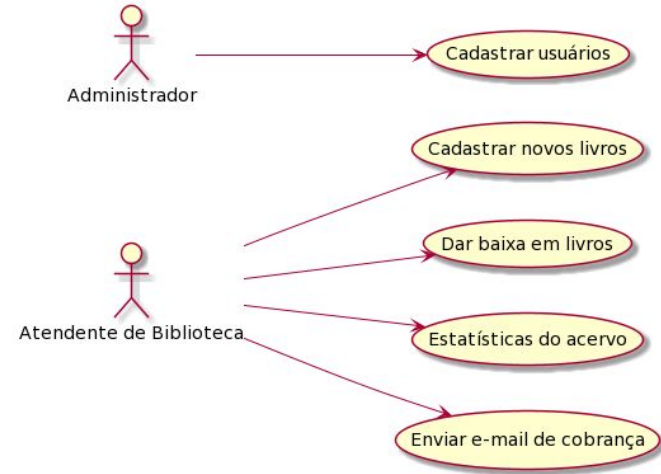
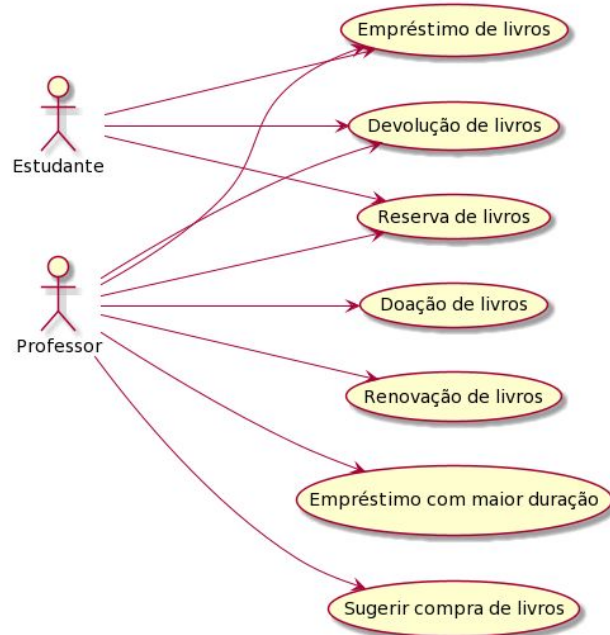
Resumo

Relações dos Casos de Uso

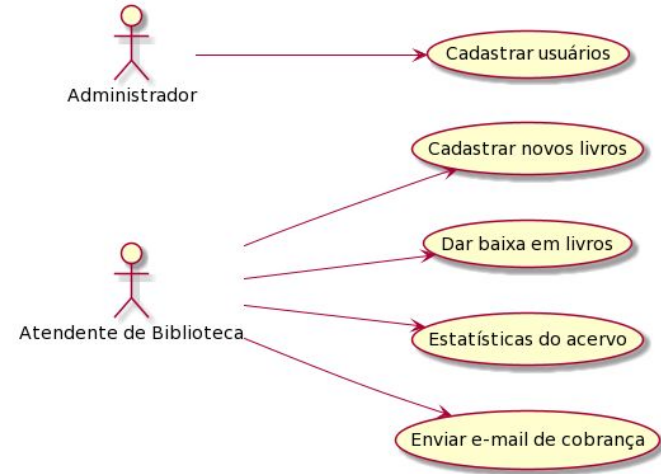
Include /
Extend



Especificação de Interações Ator → UC

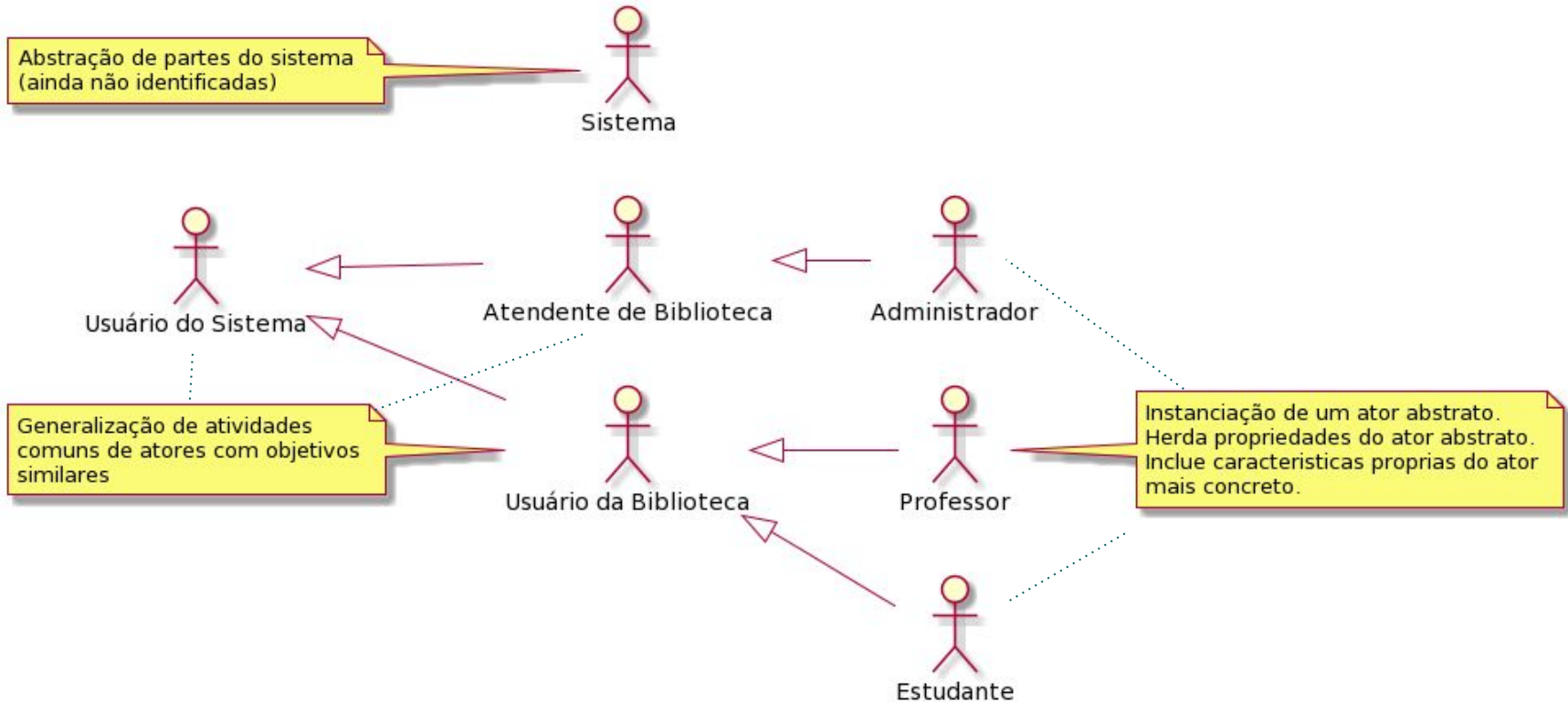


Especificação de Interações: Ator → UC



Refinando a Especificação de Interações Ator → UC

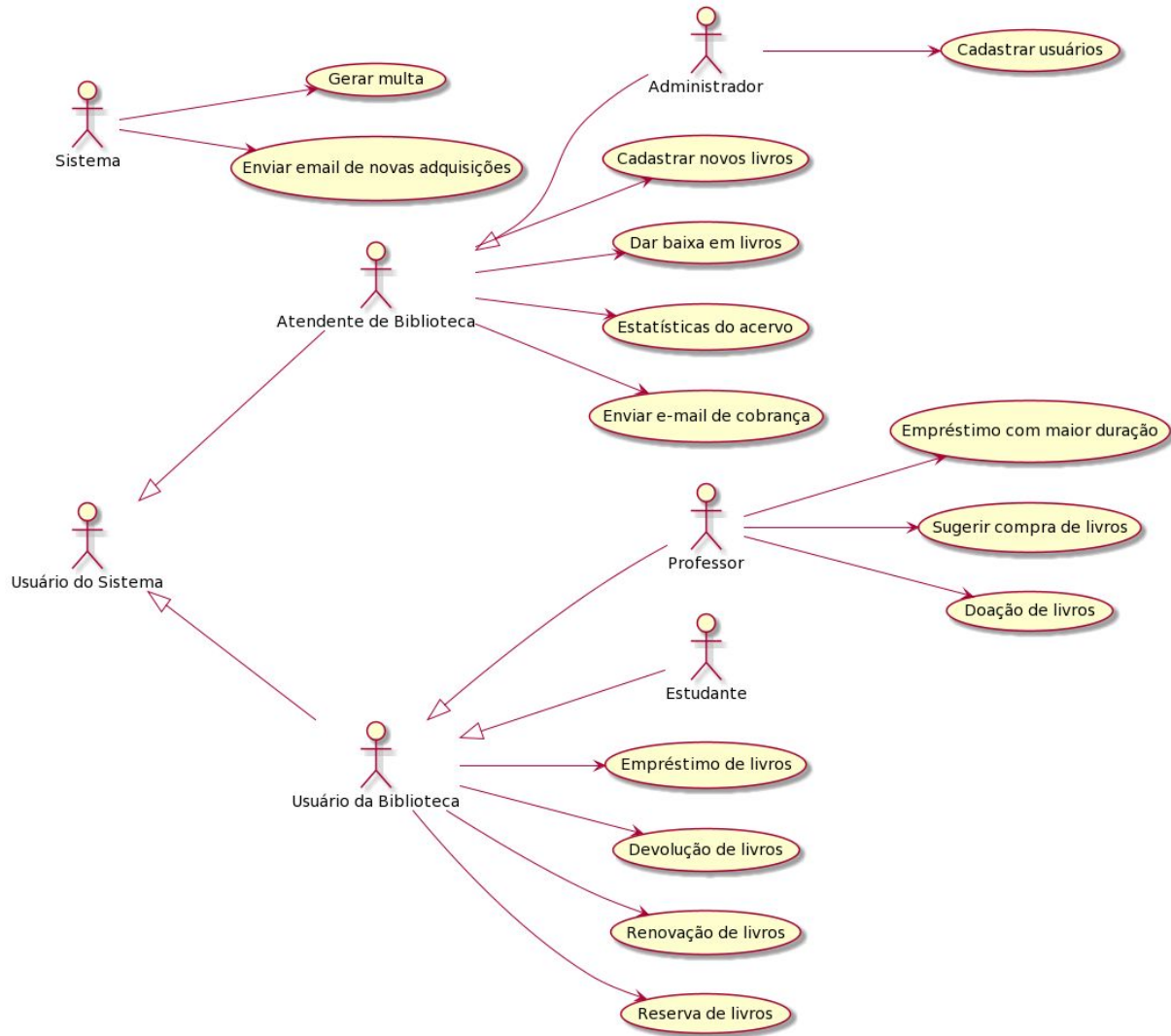
Aplicando a Relação de Herança entre atores



Refinando a Especificação de Interações Ator → UC

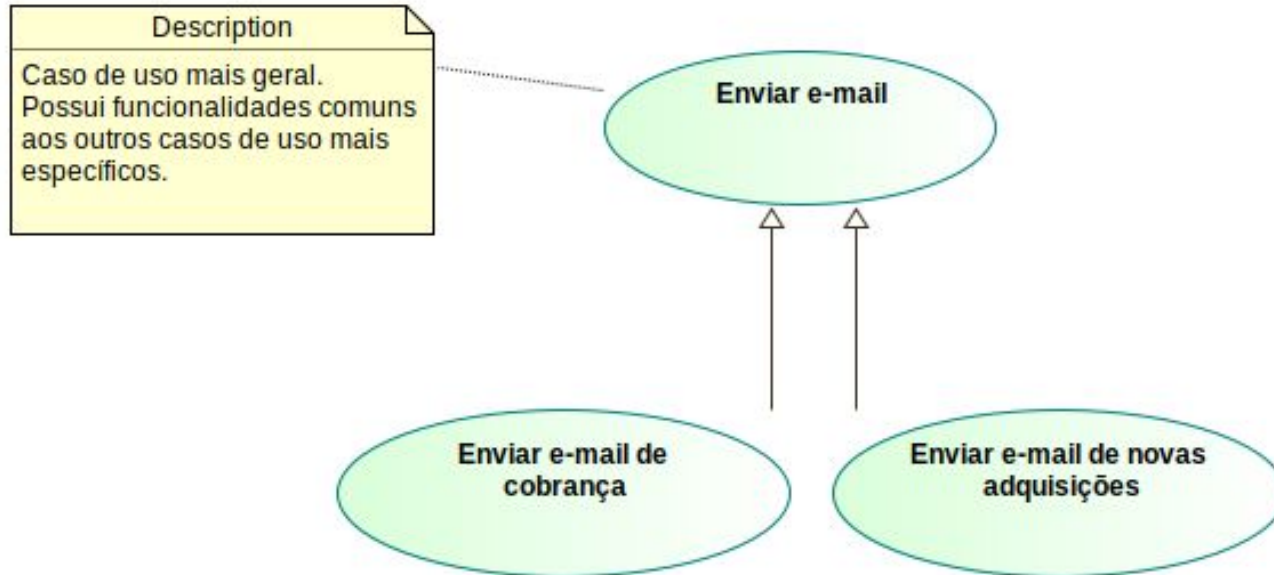
Aplicando a Relação de Herança

Princípio de OO: Abstração



Refinando a Especificação de Interações UC → UC

Aplicando a Relação de Herança entre UC



Princípio de OO: Abstração

Princípio de Design – Alta Coesão

Coesão está, na verdade, ligado ao **princípio da responsabilidade única**, que foi introduzido por Robert C. Martin no início dos anos 2000 e diz que uma entidade (UC, classe, componente, módulo) deve ter apenas **uma única responsabilidade e deve realizá-la de maneira satisfatória**, ou seja, uma entidade não deve assumir responsabilidades que não são suas. Uma vez sendo ignorado este princípio, passamos a ter problemas, como dificuldades de manutenção e de reuso no software que estamos desenvolvendo.

Cenário ideal → Aumentar a coesão das entidades no software

Princípio de Desing – Baixo Acoplamento

O acoplamento significa o quanto uma entidade (UC, classe, módulo, componente, etc) depende da outra para funcionar. E **quanto maior for esta dependência entre ambas, dizemos que estas classes elas estão fortemente acopladas.** O forte acoplamento também nos traz muitos problemas de reuso e manutenção.

Cenário ideal → Diminuir o acoplamento entre entidades no software

Refinando a Especificação de Interações Ator → UC

Aplicando a Relação de Inclusão <<Include>> e Extensão

<<Extend>>

Este tipo de relações é importante quando:

- Um UC possui um **comportamento comum** a vários outros casos de uso.
 - Aplicação do princípio de baixo acoplamento
- Vários UC (muito concretos) podem ser **compostos (agregados)** para formar um UC que execute um **processo bem definido**, com começo, meio e fim
 - Aplicação do princípio de Alta Coesão
- Um UC é muito **complexo de entender** sendo necessária sua decomposição em UC mais fáceis de entender e especificar.
 - Aplicação do princípio de Alta Coesão

Benefícios:

- Evitar repetir comportamento → aumento do reúso
- Melhora a compreensão do caso de uso → facilita manutenção

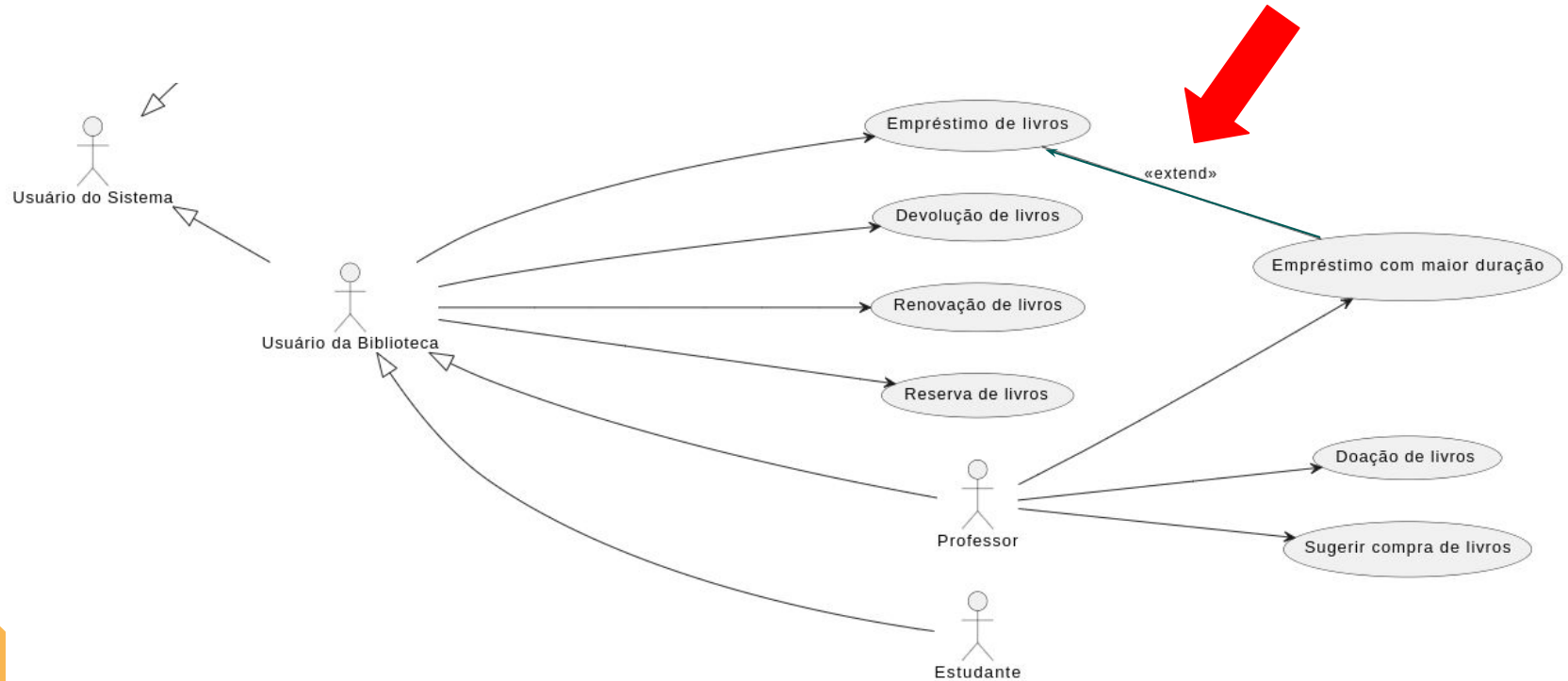
Atenção!!

O relacionamento <<Include>> entre dois casos de uso **OBRIGA a execução** do UC incluído quando o UC base é executado

O relacionamento <<Extend>> entre dois casos de uso **NÃO obriga a execução** do UC incluído quando o UC base é executado. A execução do UC estendido é **OPCIONAL**

Aplicando a Relação de Extensão <<Extend>>

Identificação de UC com comportamentos comuns



Aplicando a Relação de Inclusão <<Include>>

Identificação de UC com
comportamentos comuns

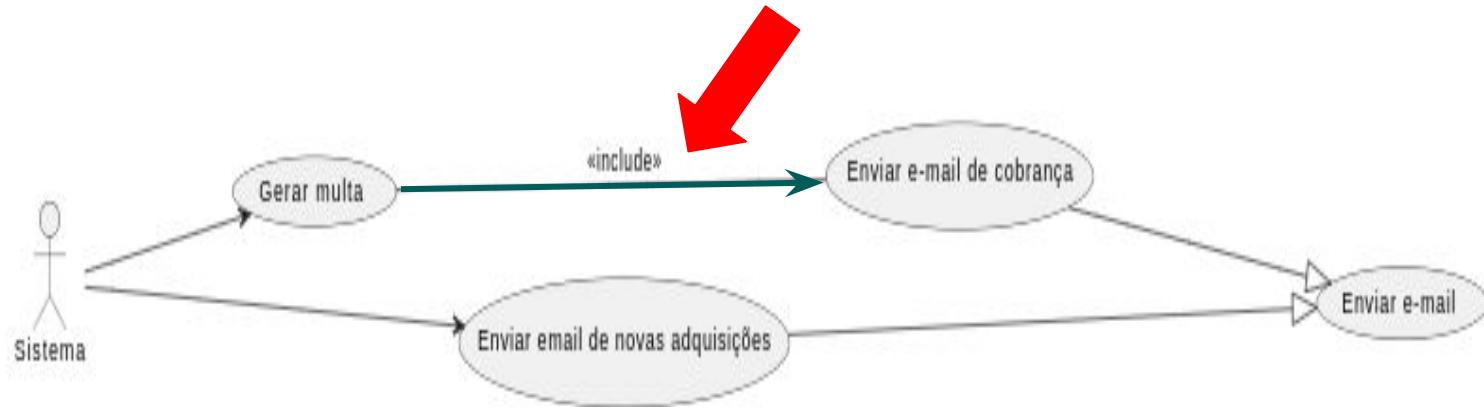


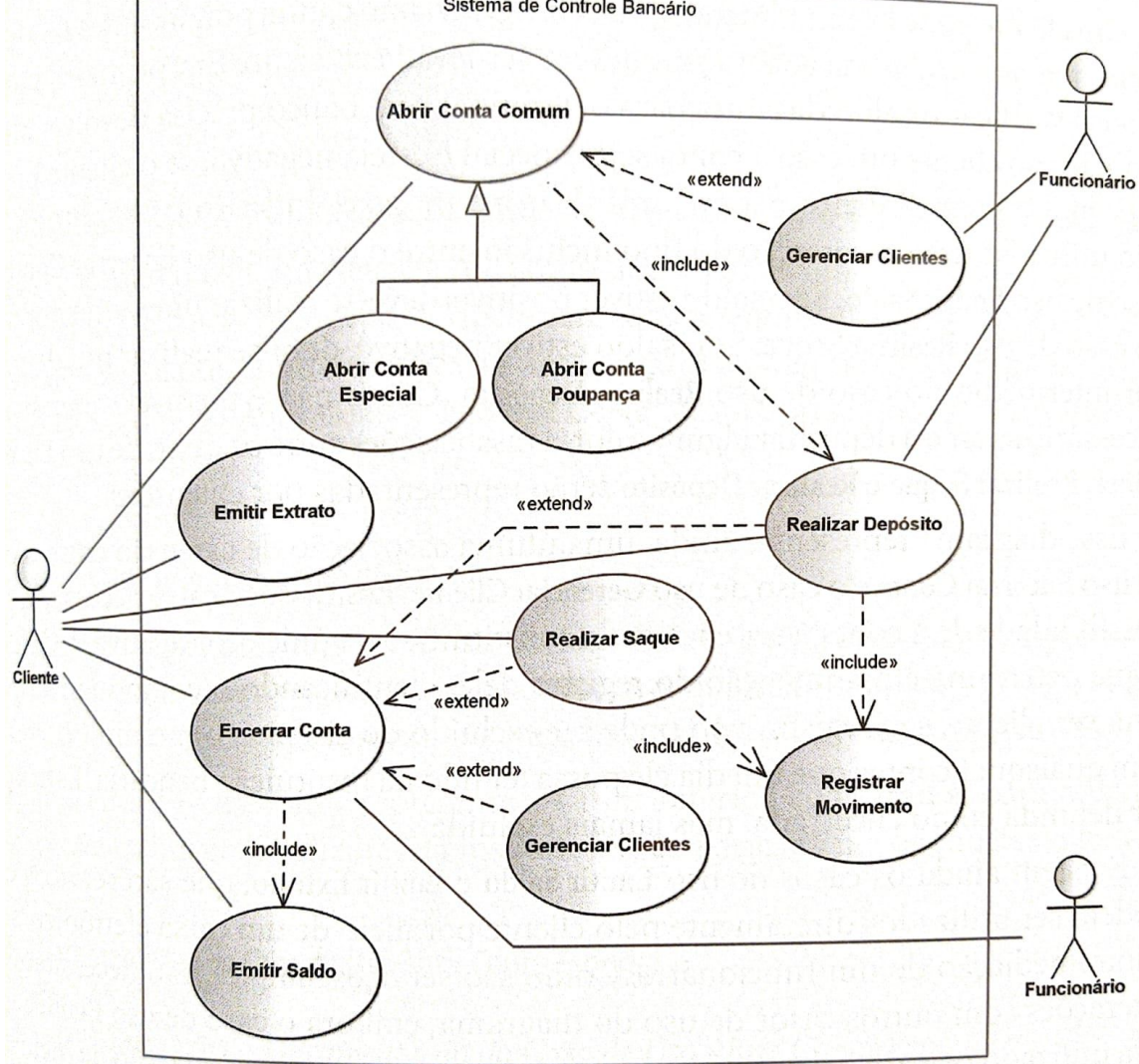
Diagrama de Casos de Uso

Sistema de Biblioteca



Diagrama de Casos de Uso

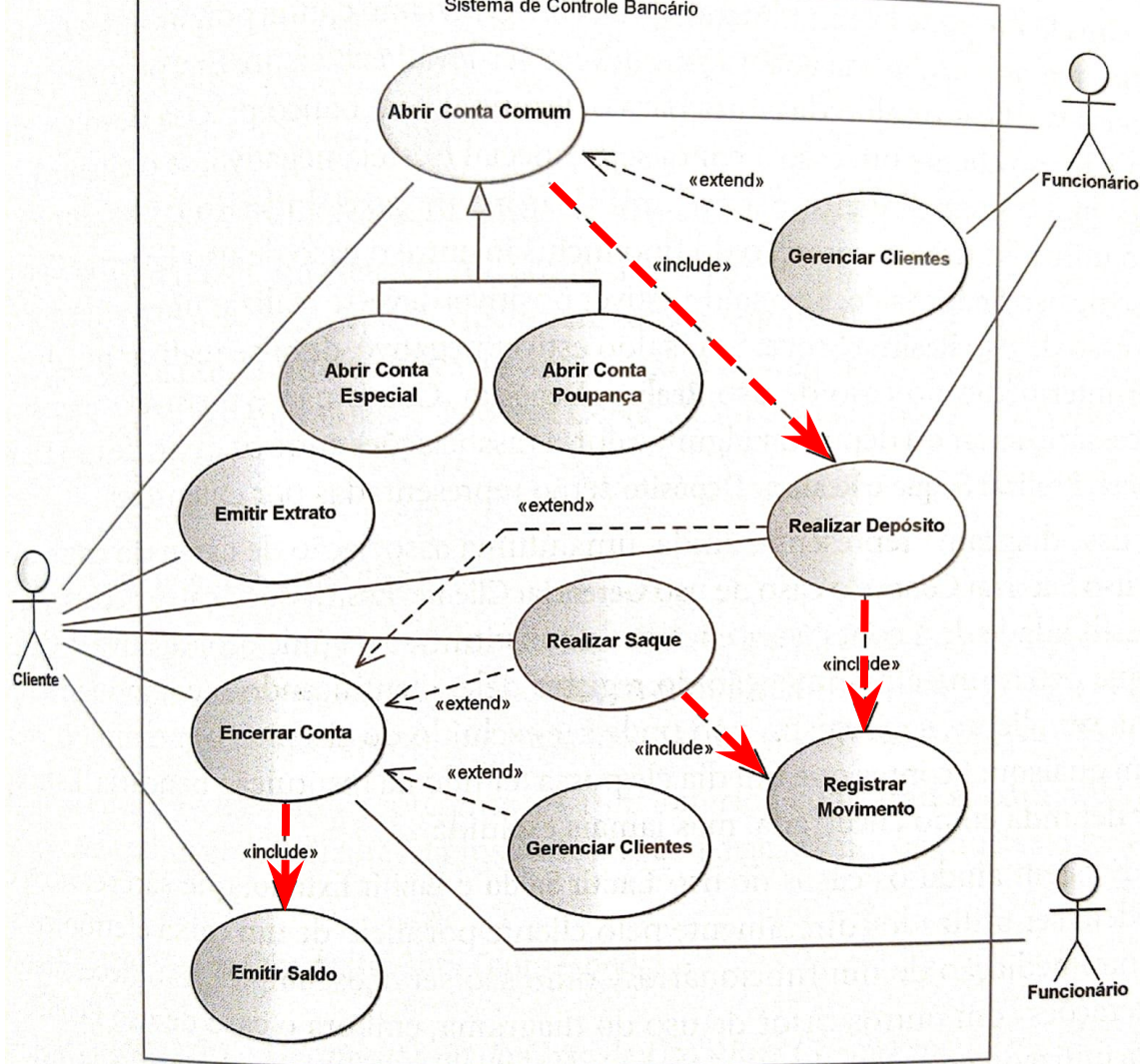
Sistema de Controle Bancário



Aplicando a Relação de
Inclusão <<Include>>

Diagrama de Casos de Uso

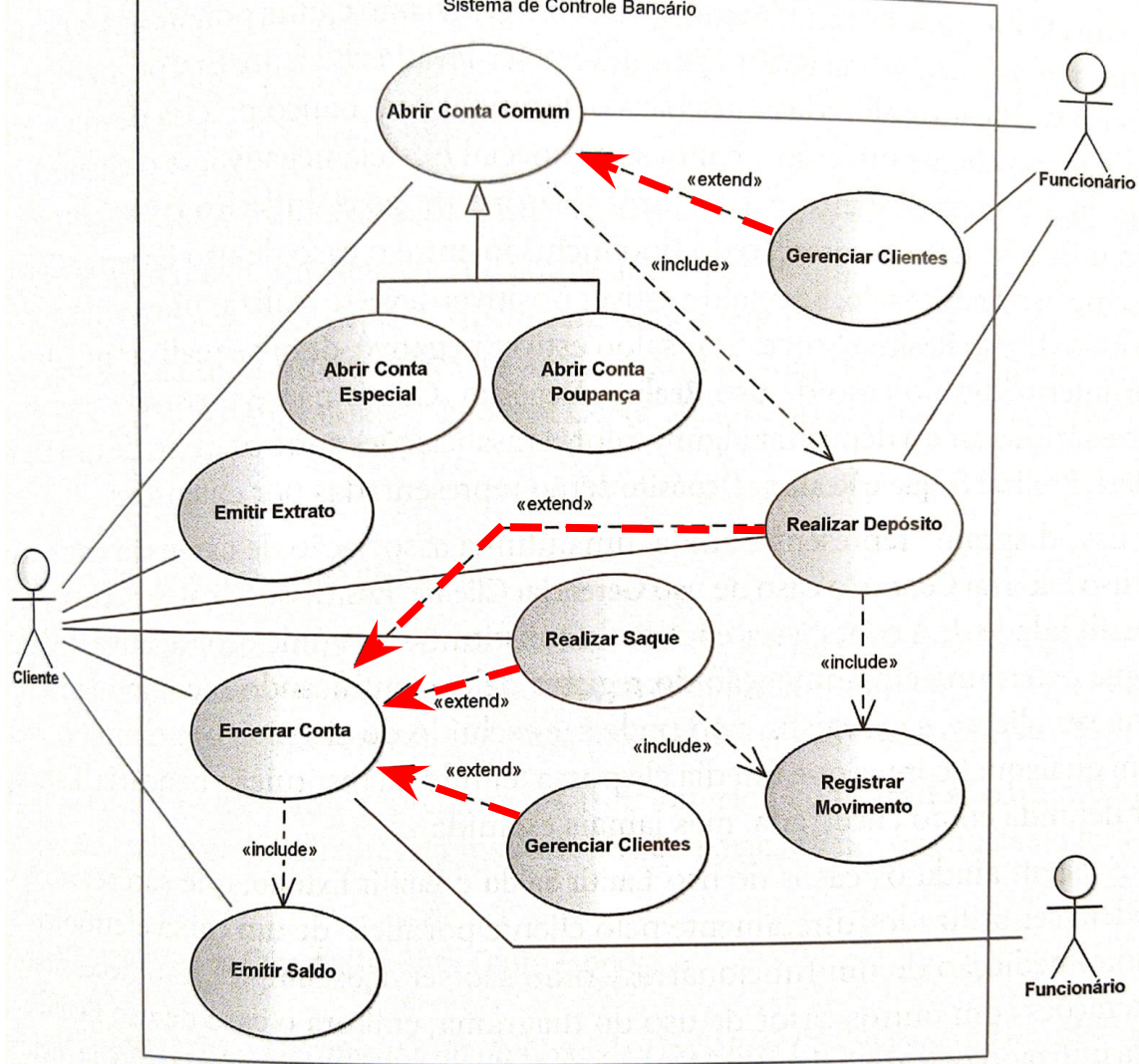
Sistema de Controle
Bancário



Aplicando a Relação de Extensão <<Extend>>

Diagrama de Casos de Uso

Sistema de Controle Bancário



Referências

Capítulos 5 e 6 do Livro: Pressman, Roger S. Engenharia de software: uma abordagem profissional. 8a ed. Porto Alegre: Bookman/Amg Editora, 2016.

Capítulo 4 do Livro: Sommerville, Ian. Engenharia de software. 9a ed. São Paulo. Pearson Prentice-Hall, 2011.

Capítulo 3 do Livro: Marco Tulio Valente. Engenharia de Software Moderna. Princípios e práticas para desenvolvimento de software com produtividade. 2019. Disponível online: <https://engsoftmoderna.info/cap3.html>

PERSONAS → <http://www.agilemodeling.com/artifacts/personas.htm>

Capítulo 6. Process. Livro: Grady Booch, robert A. Maksymchuk, Michael W. Engle, Bobbi J. Young, Jin Conallen, Kelli A.Houston. Object-Oriented Analysis and Design with Applications. 3rd Ed. 2007.

ISO/IEC 25010:2011 [ISO/IEC 25010:2011]. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models On-line: <https://www.iso.org/standard/35733.html>

Len Bass, Paul Clements, and Rick Kazman. 2012. Software Architecture in Practice (3rd. ed.). Addison-Wesley Professional.